

Separating Pseudorandom Generators from Logarithmic Pseudorandom States

Mohammed Barhoush

Université de Montréal (DIRO), Montréal, Canada
mbarhoush18@gmail.com

Abstract. Pseudorandom generators (PRGs) are a foundational primitive in classical cryptography, underpinning a wide range of constructions. In the quantum setting, pseudorandom quantum states (PRSs) were proposed as a potentially weaker assumption that might serve as a substitute for PRGs in cryptographic applications. Two primary size regimes of PRSs have been studied: logarithmic-size and linear-size. Interestingly, logarithmic PRSs have led to powerful cryptographic applications, such as digital signatures and quantum public-key encryption with tamper-resilient keys, that have not been realized from their linear counterparts. However, PRGs have only been black-box separated from linear PRSs, leaving open the fundamental question of whether PRGs are also separated from logarithmic PRSs.

In this work, we resolve this open problem. We establish a quantum black-box separation between (quantum-evaluable) PRGs and PRSs of either size regime. Specifically, we construct a unitary quantum oracle with inverse access relative to which no black-box construction of PRG from (logarithmic or linear) PRS exists.

This does not directly separate PRG from some of the applications of SPRS since these applications involve, as a first step, a non-black-box construction of a notion termed $\perp$ -PRGs. To address this, we present another unitary separation showing that PRG are also separated from $\perp$ -PRGs. Thus, we obtain separation from digital signatures and quantum public-key encryption.

1 Introduction

Pseudorandom quantum states (PRS) were introduced by Ji, Liu, and Song [14] as the quantum analogue of classical pseudorandom generators (PRG) [6]. Informally, an n -PRS generator is a quantum polynomial-time algorithm that, on input $k \in \{0, 1\}^\lambda$, outputs a n -qubit state $|\phi_k\rangle$ that is computationally indistinguishable from a Haar-random state, even when an adversary is given polynomially many copies of the state.

Two size regimes of PRS have emerged as central: (1) short pseudorandom states (SPRS), where the output length n is logarithmic in λ , and (2) long pseudorandom states (LPRS), where n scales linearly with λ . These regimes display markedly different behavior. In contrast to PRGs, no known reduction connects SPRS and LPRS. In fact, a separation exists indicating $\text{LPRS} \not\rightarrow \text{SPRS}$ [12], and there is evidence that the reverse direction is also difficult [7].

Both SPRS and LPRS can be realized from PRGs [14, 9]. In the other direction, Kretschmer [15] proved a black-box separation between PRGs and LPRSs,¹ yet this does not rule out a reduction to SPRSs. His argument relies crucially on a concentration inequality for quantum states (Theorem 5.17 in [18]) that fails in the short-state regime. Our approach does not use this result, thereby circumventing that limitation.

The difference between the two regimes is also reflected in their cryptographic power. Long pseudorandom states enable several important primitives such as quantum pseudo-encryption, quantum bit-commitment protocols, length-restricted one-time signatures with quantum public keys, and private-key quantum money [14, 2, 19]. Nevertheless, there remains a significant gap between the cryptographic power of LPRSs and PRGs.

Short pseudorandom states, in contrast, exhibit a much tighter connection to classical pseudorandomness: through quantum state tomography, one can extract a classical pseudorandom string from SPRSs [1]. However, tomography is inherently probabilistic, leading to a notion termed *pseudodeterministic* PRGs—generators that are deterministic on most inputs but may behave nondeterministically on an inverse-polynomial fraction.² Another issue to note is that this is a non-black-box construction! Nevertheless, this conversion enables many of the applications of PRGs, such as statistically binding commitments with classical communication.

While pseudodeterministic PRGs proved useful, their nondeterminism complicates their use as direct substitutes for standard PRGs in some applications. To mitigate this, Barhoush et al. [3] introduced the notion of $\perp$ -pseudodeterministic pseudorandom generators ($\perp$ -PRGs), which they build from pseudodeterministic PRGs by marking nondeterministic outcomes with the symbol $\perp$. This seemingly modest modification provides a clean interface for cryptographic constructions

¹ The separation is shown relative to quantum-evaluable one-way functions. Since any PRG is also one-way, the result extends to PRGs as well.

² Different notions of pseudodeterminism appear in the literature [5, 3]; we follow the definition of [1].

to handle nondeterministic behavior explicitly. As a result, it enables SPRS to realize applications traditionally achievable only through standard PRGs—such as many-time digital signatures and quantum public-key encryption with tamper-resilient keys [1,3]—capabilities still beyond reach for LPRSs.

Despite these advances, no complete quantum fully black-box (BB) separation is currently known between PRGs and SPRSs or even PRGs and $\perp$ -PRGs, leaving the precise relationship between them unresolved and somewhat limiting the significance of these applications.

A resolution of this question would shed light on the fundamental limits of quantum pseudorandomness and clarify which cryptographic primitives can be realized without assuming full PRG. While partial progress has been made (see Section 1.2), a complete BB separation between PRGs and SPRSs or $\perp$ -PRGs remains open. The importance of this problem has been repeatedly emphasized in recent works [3,1,4], underscoring its role in understanding the hierarchy of assumptions within the emerging framework of MicroCrypt quantum primitives.

1.1 Our Results

In this work, we resolve this issue by establishing two black-box separations. First, we separate PRGs from ℓ -PRSs of any size $\ell \in O(\lambda)$, thereby also ruling out reductions to SPRS. Second, we separate SPRS from $\perp$ -PRGs. Both separations hold relative to a unitary quantum oracle with inverse access, capturing a broad class of black-box constructions [12].

Theorem 1. *There does not exist a BB construction of a PRG from an ℓ -PRS (even with inverse access), for any function $\ell(\lambda) \in O(\lambda)$.*

We consider a general notion of PRGs that is quantum-evaluable and permits a negligible probability of error or non-determinism.

Moreover, prior works have shown that SPRSs imply a rich class of cryptographic primitives. These applications proceed in three conceptual steps:

1. Construct pseudodeterministic PRGs from SPRS [1] (*non-BB*).
2. Construct $\perp$ -PRG from pseudodeterministic PRGs [3] (*BB*).
3. Use $\perp$ -PRGs to achieve some of the same applications achieved with PRG such as digital signatures [1,3] (*BB*).

The first step, unusually, is a non-BB construction: the correctness of the pseudodeterministic PRG is guaranteed only by the security of the underlying SPRS. Hence, the construction violates a key requirement of BB reductions—namely, that the correctness of the target primitive must follow from the correctness of the source. Consequently, separating PRGs from SPRSs alone is insufficient to separate PRGs from pseudodeterministic PRGs and their many derived applications.

We present another unitary separation to resolve this issue.

Theorem 2. *There does not exist a BB construction of a PRG from an $\perp$ -PRG (even with inverse access).*

Recall, $\perp$ -PRGs can be used to construct digital signatures and quantum public-key encryption [1,3]. As a result, we obtain the following as a direct corollary.

Corollary 1. *There does not exist a BB construction of a PRG from any of the following primitives:*

1. *(Many-time) existentially unforgeable digital signatures for classical messages with classical keys and signatures (DS).*
2. *CPA-secure quantum public-key encryption of classical messages with tamper-resilient keys and classical ciphertexts (QPKE).*

We note that a weaker form of Theorem 2 and Corollary 1 was previously established by [4], but only for completely positive trace-preserving (CPTP) maps (see Section 1.2). However, CPTP separations exclude a narrower class of constructions than unitary separations. In particular, a CPTP separation does not rule out reductions that rely on unitary access to the underlying primitive or adversary (see [12,4] for further discussion of these distinctions). In contrast, Theorem 2 and Corollary 1 establish, for the first time, general separations between these fundamental cryptographic primitives, encompassing reductions that may employ unitary or inverse access.

1.2 Related Works

Ananth, Lin, and Yuen [1] were the first to formally investigate applications specific to SPRSs. They showed how to construct pseudodeterministic PRGs from SPRSs, and used them to obtain applications such as statistically binding commitments with classical communication and CPA-secure symmetric encryption with classical ciphertexts.

A follow-up work by Barhoush et al. [3] introduced the notion of $\perp$ -PRGs, which they build from pseudodeterministic PRGs, and use to achieve applications such as DS and QPKE.

On the negative side, there are several separations that are relevant to this work:

- (LPRS $\not\Rightarrow$ PRG) : Kretschmer [15] presented a separation between PRGs and LPRSs, implying that LPRS constitute a potentially weaker assumption than PRGs.
- (LPRS $\not\Rightarrow$ DS) : Coladangelo and Mutreja [12] showed that even digital signatures are separated from LPRS.
- (LPRS $\not\Rightarrow$ SPRS) : This is a direct corollary to the separation above by [12], but it was also shown through an alternative approach by [8].
- ($\perp$ -PRG $\not\Rightarrow$ OWSG) : Barhoush, Nishimaki, and Yamakawa [4] showed that one-way state generators (OWSG) are separated from $\perp$ -PRGs, relative to a non-unitary CPTP map. Here, OWSGs are the quantum analogues of one-way functions (OWF) and every OWF is a OWSG. In comparison, our result is based on a unitary oracle and separates PRGs from $\perp$ -PRGs. In one sense, this improves on [4], as our separation is unitary; yet, because PRGs also imply OWSGs, the two separations are not directly comparable.

- (SPRS $\not\rightarrow$ PRG) : At the same time as our work, Bouaziz–Ermann, Hhan, Muguruza, and Vu [7] introduced a novel conjecture on product states such that, if it holds, PRGs and SPRSs can be separated under a different approach than our own. Our result strengthens their separation as we do not assume any conjectures.
- (quantum-evaluable OWF $\not\rightarrow$ classical-evaluable OWF) : Kretschmer, Qian, Tal [16] show a separation between classical-evaluable OWFs and quantum-evaluable OWFs relative to a classical oracle. As a corollary, they establish that classically-evaluable OWFs are separated from DS and QPKE. However, our results imply that even quantum-evaluable OWFs are separated from these applications. Note that all notions in our work consider quantum-evaluable algorithms.

2 Technical Overview

We now give an overview of our two separations.

2.1 Separating PRG from SPRS

We describe our separation between PRGs and ℓ -PRSs for any function $\ell(\lambda) \in O(\lambda)$. The discussion below presents a high-level overview that omits several technical subtleties appearing in the full proof.

We use two unitary oracles in the separation:

1. A PSPACE oracle $\mathcal{C}$, which is used to break any PRG.
2. A ℓ -Common Haar Function-Like State (CHFS) oracle [7] $\mathcal{O}$, which, on an input $x \in \{0, 1\}^*$, outputs a Haar random state $|\phi_x\rangle$ of size $\ell(|x|)$, where $|x|$ denotes the length of x .

It is not difficult to show that $\mathcal{O}$ acts as a PRS, and this was established earlier in [15].

Suppose, toward a contradiction, that there exists a BB construction of a PRG from a PRS. This would imply the existence of a PRG $G^{\mathcal{O}}$ in this oracle model that may query $\mathcal{O}$. Because all CHFS oracles $\mathcal{O}$ satisfy PRS correctness, the resulting PRG G must also satisfy correctness for any oracle—that is, for every oracle $\mathcal{O}$, except with negligible probability, on an input k , the evaluation $G^{\mathcal{O}}(k)$ must produce a fixed output.

Now consider two CHFS oracles $\mathcal{O}$ and $\hat{\mathcal{O}}$. For any $\lambda \in \mathbb{N}$, define the set of disagreement inputs:

$$\text{Disagr}_{\lambda}^{\mathcal{O}, \hat{\mathcal{O}}} = \left\{ k \in \{0, 1\}^{\lambda} \mid G^{\mathcal{O}}(k) \neq G^{\hat{\mathcal{O}}}(k) \right\}. \quad (1)$$

Intuitively, two situations can arise:

1. There exists a sufficiently large λ such that for any pair of oracles $\mathcal{O}, \hat{\mathcal{O}}$, the set $\text{Disagr}_{\lambda}^{\mathcal{O}, \hat{\mathcal{O}}}$ consists of only a negligible fraction of inputs.

2. There exists an infinite sequence $(\lambda_j)_{j \in \mathbb{N}}$ such that for each j , there exists a pair of oracles $\mathcal{O}^j, \hat{\mathcal{O}}^j$ such that the set $\text{Disagr}_{\lambda_j}^{\mathcal{O}^j, \hat{\mathcal{O}}^j}$ consists of an inverse-polynomial fraction of inputs.

If the first case holds for all pairs of oracles, then $G^{\mathcal{O}}$ is essentially independent of $\mathcal{O}$ for large enough λ and exists without oracle access. However, it is well known that no such PRG can exist relative to a PSPACE oracle; thus, this case leads to a contradiction.

We therefore focus on the second case: there exist an infinite sequence $(\lambda_j)_{j \in \mathbb{N}}$ such that, for each j , there exists a pair of oracles $(\mathcal{O}^j, \hat{\mathcal{O}}^j)$ such that the set $\text{Disagr}_{\lambda_j}^{\mathcal{O}^j, \hat{\mathcal{O}}^j}$ consists of a inverse-polynomial fraction of inputs.

Assume that G makes at most $T = T(\lambda)$ oracle queries for some polynomial T , and define $m = \pi \cdot T \cdot \lambda$. For each j , we construct a sequence of intermediate oracles $\mathcal{O}_1^j, \dots, \mathcal{O}_m^j$, starting with $\mathcal{O}_1^j = \mathcal{O}^j$ and ending with $\mathcal{O}_m^j = \hat{\mathcal{O}}^j$, such that for every input state ρ ,

$$\text{Tr}(\mathcal{O}_i^j(\rho), \mathcal{O}_{i+1}^j(\rho)) \leq \frac{\pi}{m}. \quad (2)$$

To build this sequence, we gradually “move” all the output states of $\mathcal{O}^j$ toward that of $\hat{\mathcal{O}}^j$, reducing the trace distance by at most π/m at each step. After at most m steps, we reach $\hat{\mathcal{O}}^j$. We essentially consider the shortest geodesic path between every two states on the spherical representation of the states.

For any $i \in [m]$, the trace distance between the responses of $\mathcal{O}_i^j$ and $\mathcal{O}_{i+1}^j$ on any query is bounded by π/m . Since G makes at most T oracle queries, it cannot distinguish between $\mathcal{O}_i^j$ and $\mathcal{O}_{i+1}^j$ with probability exceeding $T/m < 1/\lambda_j$. Therefore, for any input $k \in \{0, 1\}^{\lambda_j}$,

$$\text{Tr}(G^{\mathcal{O}_i^j}(k), G^{\mathcal{O}_{i+1}^j}(k)) \leq \frac{1}{\lambda_j}. \quad (3)$$

Now consider an input k such that $y_k = G^{\mathcal{O}^j}(k)$ and $\hat{y}_k = G^{\hat{\mathcal{O}}^j}(k)$ are distinct with high probability. As we progress along the oracle sequence $\mathcal{O}_1^j, \dots, \mathcal{O}_m^j$, the probability that $G^{\mathcal{O}_i^j}(k)$ outputs y_k must gradually decrease, while the probability of outputting $\hat{y}_k$ must gradually increase. Hence, there must exist some index i_k such that $G^{\mathcal{O}_{i_k}^j}(k)$ is non-deterministic.

At this point, no direct contradiction arises yet, since G may be non-deterministic on a negligible fraction of inputs. However, we have established that there exists a non-negligible fraction of inputs on which $G^{\mathcal{O}^j}$ and $G^{\hat{\mathcal{O}}^j}$ disagree. For each such input, some generator in the sequence $G^{\mathcal{O}_1^j}, \dots, G^{\mathcal{O}_m^j}$ must be non-deterministic on that input. Because m is polynomial while the fraction of such inputs is non-negligible, there must exist an index i_j for which $G^{\mathcal{O}_{i_j}^j}$ is non-deterministic on a non-negligible fraction of inputs.

Still, this does not immediately violate correctness, since $G^{\mathcal{O}_{i_j}^j}$ might be non-deterministic only for the specific parameter λ_j and fully deterministic for all

larger $\lambda > \lambda_j$. Nevertheless, we have established that for every j , there exists an oracle $\mathcal{O}_{i_j}^j$ such that $G^{\mathcal{O}_{i_j}^j}$ behaves non-deterministically on security parameter λ_j .

Next, we “combine” the oracles $\mathcal{O}_{i_j}^j$ for all $j \in \mathbb{N}$ into a single oracle $\overline{\mathcal{O}}$ such that, for infinitely many λ , $G^{\overline{\mathcal{O}}}$ is non-deterministic on a non-negligible fraction of inputs. This yields a contradiction: any valid BB construction must preserve correctness whenever the underlying primitive does, and PRSs in this oracle model satisfy correctness for all oracles, including $\overline{\mathcal{O}}$.

A final subtlety arises because G may always query its oracle on small inputs, independent of the security parameter. In that case, it becomes unclear how to combine oracles that differ on small inputs. To address this, we modify G so that it queries $\mathcal{O}$ only on inputs of length at least $\log(\lambda)$; for smaller inputs, G can internally simulate $\mathcal{O}$ using its own randomness, following the simulation approach of [10]. Then, as long as the sequence $(\lambda_j)_{j \in \mathbb{N}}$ is chosen to be sparse enough, the queries of G under different security parameters λ_j do not intersect and we can combine the oracles to obtain a contradiction.

2.2 Separating PRG from $\perp$ -PRG

We now describe how to separate PRG from $\perp$ -PRG using a unitary quantum oracle with inverse access. In particular, we use two oracles:

1. A PSPACE oracle, used to break any PRG.
2. A modified quantum random oracle with an abort mechanism $\mathcal{O}$, designed to satisfy the pseudodeterminism property of $\perp$ -PRGs. For an inverse-polynomial fraction of inputs (say $1/\lambda$), $\mathcal{O}$ outputs either a fixed string or the symbol $\perp$; for all remaining inputs, it behaves deterministically, outputting a fixed string only.

It is straightforward to show that $\mathcal{O}$ is a valid $\perp$ -PRG. Suppose, toward a contradiction, that there exists a BB construction of a PRG from a $\perp$ -PRG. This would imply the existence of a PRG $G^{\mathcal{O}}$.

The proof follows a similar template as the one above. We establish the existence of an infinite sequence $(\lambda_j)_{j \in \mathbb{N}}$ such that, for each j , there exists a pair of $\perp$ -random oracles $(\mathcal{O}^j, \hat{\mathcal{O}}^j)$ such that the set $\text{Disagr}_{\lambda_j}^{\mathcal{O}^j, \hat{\mathcal{O}}^j}$ consists of an inverse-polynomial fraction of inputs.

For each j , we construct a sequence of intermediate oracles $\mathcal{O}_1^j, \dots, \mathcal{O}_n^j$ (for some polynomial n), starting with $\mathcal{O}_1^j = \mathcal{O}^j$ and ending with $\mathcal{O}_n^j = \hat{\mathcal{O}}^j$, such that any pair of consecutive oracles cannot be distinguished except with small probability i.e. they satisfy a condition similar to Eq. (2).

Unlike in the PRS-based separation, we cannot incrementally move all the outputs since each oracle must map most inputs to a single fixed classical output. Instead, we modify the behavior on a small fraction (roughly $1/\lambda$) of inputs while keeping the rest deterministic.

Concretely, for an input x where $\mathcal{O}^j(x)$ outputs either $\perp$ or y_x and $\hat{\mathcal{O}}^j(x)$ outputs $\hat{y}_x$, we gradually increase the probability that $\mathcal{O}^j(x)$ outputs $\perp$ until this

occurs with probability 1, and then slowly increase the probability of outputting $\hat{y}_x$ until it reaches 1. Throughout this sequence, each oracle maps x to either a fixed value or $\perp$, and at most a $3/\lambda$ fraction of all inputs yield $\perp$.

We then repeat this modification on a fresh set of $1/\lambda$ inputs. Every oracle in the resulting sequence remains a valid $\perp$ -PRG (with a slightly larger pseudodeterminism error). Yet, G cannot distinguish between consecutive oracles except with small probability.

By the same reasoning as in the previous separation, there must exist an index i_j such that $G^{O_{i_j}^j}$ is non-deterministic on a non-negligible fraction of inputs. Combining these oracles across infinitely many parameters again yields a single oracle relative to which G violates correctness on infinitely many security parameters. This contradicts the requirement that any valid black-box construction must preserve correctness, completing the separation between PRG and $\perp$ -PRG.

Acknowledgments: We would like to thank Samuel Bouaziz–Ermann for discussions that helped us realize that the construction of pseudodeterministic PRGs from SPRSs in [1] is not BB.

3 Preliminaries

3.1 Notation

We let $[n] = \{1, 2, \dots, n\}$ and let $\text{negl}(x)$ denote any function that is asymptotically smaller than the inverse of any polynomial.

We let $x \leftarrow X$ denote that x is chosen from the values in X , according to the distribution X . If X is a set, then $x \leftarrow X$ simply means x is chosen uniformly at random from the set. We let $\Pi_{m,n} = (\{0, 1\}^n)^{\{0,1\}^m}$ denote the set of functions mapping $\{0, 1\}^m \rightarrow \{0, 1\}^n$.

We refer the reader to [20] for a detailed exposition to preliminary quantum information. We let $\mathcal{S}(\mathcal{H})$ and $\mathcal{U}(\mathcal{H})$ denote the set of unit vectors and unitary operators, respectively, on the Hilbert space $\mathcal{H}$ and let $\text{Haar}(\mathbb{C}^d)$ denote the Haar measure over $\mathbb{C}^d$ which is the uniform measure over all d -dimensional unit vectors. We let Tr denote the total trace distance between two density matrices or two distributions.

We follow the standard notations to define quantum algorithms. We say that a quantum algorithm A is *QPT* if it consists of a family of quantum algorithms $\{A_\lambda\}_\lambda$ such that the run-time of each algorithm A_λ is bounded by some polynomial $p(\lambda)$. We say that $A_\lambda(x)$ is *non-deterministic* if evaluating $A_\lambda(x)$ twice yields distinct values with non-negligible probability. For a constant $c \in (0, 1)$, we say that $A_\lambda(x)$ is *c-non-deterministic* if evaluating $A_\lambda(x)$ twice yields distinct values with at least c probability. We avoid using the λ subscript in algorithms to avoid excessive notation.

3.2 Pseudorandom Primitives

We define pseudorandom states (PRSs), first introduced in [14].

Definition 1 (Pseudorandom State Generator). *Let $\lambda \in \mathbb{N}$ be the security parameter and let $n = n(\lambda)$ be a function in λ . A QPT algorithm PRS is called a n -pseudorandom state generator (PRS) if the following holds:*

- (Correctness) *On any input $k \in \{0, 1\}^\lambda$, $\text{PRS}(k)$ outputs a n -qubit state.*
- (Security) *For any polynomial $t(\cdot)$ and QPT distinguisher $\mathcal{A}$:*

$$\left| \Pr_{k \leftarrow \{0, 1\}^\lambda} [\mathcal{A}(\text{PRS}(k)^{\otimes t(\lambda)}) = 1] - \Pr_{|\phi\rangle \leftarrow \text{Haar}(\mathbb{C}^n)} [\mathcal{A}(|\phi\rangle^{\otimes t(\lambda)}) = 1] \right| \leq \text{negl}(\lambda).$$

We divide PRS into two regimes, based on the state size n :

1. $n = \Theta(\log(\lambda))$, which we call short pseudorandom states (SPRSs).
2. $n = \Theta(\lambda)$, which we call long pseudorandom states (LPRSs).

We will also recall the standard definition for PRGs.

Definition 2 (Pseudorandom Generator). *Let $\lambda \in \mathbb{N}$ be the security parameter and let $n = n(\lambda)$ be polynomial in λ . A QPT algorithm G is called a n -pseudorandom generator (PRG), if*

- (Expansion) $n > \lambda$ for all $\lambda \in \mathbb{N}$.
- (Correctness) *For any input $k \in \{0, 1\}^\lambda$, there exists a string $y_k \in \{0, 1\}^n$ such that the following holds over the distribution of inputs,*

$$\Pr_{k \leftarrow \{0, 1\}^\lambda} [G(k) = y_k] \geq 1 - \text{negl}(\lambda). \quad (4)$$

- (Security) *For any QPT distinguisher $\mathcal{A}$:*

$$\left| \Pr_{k \leftarrow \{0, 1\}^\lambda} [\mathcal{A}(G(k)) = 1] - \Pr_{y \leftarrow \{0, 1\}^n} [\mathcal{A}(y) = 1] \right| \leq \text{negl}(\lambda).$$

We now recall the definition of $\perp$ -PRGs from [3]. First, we define the following useful operator.

Definition 3 ($\text{ls-}\perp$). *We define the operator*

$$\text{ls-}\perp(a, b) := \begin{cases} \perp & \text{if } a = \perp \\ b & \text{otherwise.} \end{cases}$$

Definition 4 ($\perp$ -Pseudorandom Generator). *Let $\lambda \in \mathbb{N}$ be the security parameter and let $n = mn(\lambda)$ be polynomial in λ . A QPT algorithm G mapping $\{0, 1\}^\lambda$ to $\{0, 1\}^n \cup \{\perp\}$, is a (μ, n) - $\perp$ -pseudodeterministic pseudorandom generator ($\perp$ -PRG) if:*

1. (Expansion) $n(\lambda) > \lambda$ for all $\lambda \in \mathbb{N}$.

2. (Pseudodeterminism/Correctness) *There exist a constant $c > 0$ such that $\mu(\lambda) = O(\lambda^{-c})$ and for sufficiently large $\lambda \in \mathbb{N}$ there exists a set $\mathcal{G}_\lambda \subseteq \{0, 1\}^\lambda$ such that the following holds:*

(a)

$$\Pr_{k \leftarrow \{0, 1\}^\lambda} [k \in \mathcal{G}_\lambda] \geq 1 - \mu(\lambda).$$

(b) *For every $k \in \mathcal{G}_\lambda$ there exists a non- $\perp$ value $y \in \{0, 1\}^n$ such that:*

$$\Pr[G(k) = y] \geq 1 - \text{negl}(\lambda). \quad (5)$$

(c) *For every $k \in \{0, 1\}^\lambda$, there exists a non- $\perp$ value $y \in \{0, 1\}^n$ such that:*

$$\Pr[G(k) \in \{y, \perp\}] \geq 1 - \text{negl}(\lambda). \quad (6)$$

3. (Security) *For every polynomial $q = q(\lambda)$ and QPT distinguisher $\mathcal{A}$, there exists a negligible function ϵ such that:*

$$\left| \Pr \left[\begin{array}{c} k \leftarrow \{0, 1\}^\lambda \\ y_1 \leftarrow G(k) \\ \vdots \\ y_q \leftarrow G(k) \end{array} : \mathcal{A}(y_1, \dots, y_q) = 1 \right] - \Pr \left[\begin{array}{c} k \leftarrow \{0, 1\}^\lambda \\ y \leftarrow \{0, 1\}^n \\ y_1 \leftarrow \text{ls-}\perp(G(k), y) \\ \vdots \\ y_q \leftarrow \text{ls-}\perp(G(k), y) \end{array} : \mathcal{A}(y_1, \dots, y_q) = 1 \right] \right| \leq \epsilon(\lambda)$$

$\perp$ -PRGs were constructed from SPRSs in [3].

Lemma 1 (Corollary 1 [3]). *If there exists $(c \log \lambda)$ -SPRS for some constant $c > 12$, then there exists a $(O(\lambda^{-c/12+1}), \lambda^{c/12})$ - $\perp$ -PRG.*

3.3 Common Haar Function-like State Oracle

We first recall the definition of a swap unitary [11].

Definition 5. *For an n -qubit pure quantum state $|\phi\rangle$, the swap (or reflection) unitary is defined by*

$$S_{|\phi\rangle} = |0^n\rangle\langle\phi| + |\phi\rangle\langle 0^n| + I_\perp$$

where we assume w.l.o.g. that $|\phi\rangle$ is orthogonal to $|0^n\rangle$, as we can always add $|1\rangle$ to make it orthogonal. $I_\perp$ is the identity on the subspace orthogonal to $\text{span}\{|0^n\rangle, |\phi\rangle\}$.

We now define the (unitarized) *Common Haar Function-Like State (CHFS) oracle* as in [7].

Definition 6 (CHFS oracle). *Let $\ell = \ell(\lambda)$ be a function on the security parameter $\lambda \in \mathbb{N}$. We denote by $\mathcal{O}_\ell$ the distribution over the family of unitary oracles where*

- **Randomness:** Sample a $\ell(|x|)$ -qubit Haar random quantum state $|\phi_x\rangle$ for each $x \in \{0,1\}^*$ and define

$$\Phi = \{|\phi_x\rangle|1\rangle\}_{x \in \{0,1\}^*}.$$

- **Setup:** A family of oracles $\mathcal{O}^\Phi := (S_x^\Phi)_{x \in \{0,1\}^*} \leftarrow \mathcal{O}_\ell$ is chosen by randomly sampling Φ , where $S_x^\Phi := S_{|\phi_x\rangle}$ denotes the swap unitary as defined in Definition 5.
- **Query:** The oracle takes as a query a quantum state ρ_{XYZ} such that $|Y| = \ell(|X|) + 1$ and applies the unitary

$$\mathcal{O}^\Phi := \sum_{x \in \{0,1\}^{|X|}} |x\rangle\langle x|_X \otimes S_x^\Phi = \sum_{x \in \{0,1\}^{|X|}} |x\rangle\langle x|_X \otimes S_{|\phi_x\rangle},$$

on ρ_{XYZ} , where S_x is applied on the register Y .

Remark 1. For simplicity, we often discard the superscript Φ in $\mathcal{O}^\Phi$. Furthermore, for a classical string $x \in \{0,1\}^*$, we write $\mathcal{O}(x)$ to denote the query $\mathcal{O}(|x\rangle|0^{\ell(|x|)+1}\rangle)$.

3.4 Black-Box Separation

Black-box separating primitives using oracles was first considered in [13] and later formalized in the quantum setting in [12]. We briefly define the relative notions for this work.

Definition 7. A primitive P is a pair $P = (\mathcal{F}_P, \mathcal{R}_P)$ ³ where $\mathcal{F}_P$ is a set of quantum channels, and $\mathcal{R}_P$ is a relation over pairs $(G, \mathcal{A})$ of quantum channels, where $G \in \mathcal{F}_P$.

A quantum channel G is an implementation of P if $G \in \mathcal{F}_P$. If G is additionally a QPT channel, then we say that G is an efficient implementation of P . A quantum channel $\mathcal{A}$ P -breaks $G \in \mathcal{F}_P$ if $(G, \mathcal{A}) \in \mathcal{R}_P$. We say that G is a secure implementation of P if G is an implementation of P such that no QPT channel P -breaks it. The primitive P exists if there exists an efficient and secure implementation of P .

We now formalize the notion of constructions relative to an oracle.

Definition 8. We say that a primitive P exists relative to an oracle $\mathcal{O}$ if:

- There exists QPT oracle-access algorithm $G^{(\cdot)}$ such that $G^\mathcal{O} \in \mathcal{F}_P$.
- The security of $G^\mathcal{O}$ holds against all QPT adversaries with access to $\mathcal{O}$ i.e. for all QPT $\mathcal{A}$, $(G^\mathcal{O}, \mathcal{A}^\mathcal{O}) \notin \mathcal{R}_P$.

We are now ready to define the notion of fully black-box construction.

³ We can think of $\mathcal{F}_P$ to mean the “correctness” conditions of P and $\mathcal{R}_P$ to mean the “security” conditions of P .

Definition 9. A QPT algorithm $G^{(\cdot)}$ is a fully black-box construction (BB) of Q from P **with inverse access** if the following two conditions hold:

1. For every unitary implementation U of P , $G^{U, U^{-1}} \in \mathcal{F}_Q$.
2. There is a QPT algorithm $S^{(\cdot)}$ such that, for every unitary implementation U of P , every adversary $\mathcal{A}$ that Q -breaks $G^{U, U^{-1}}$, and every unitary implementation $\tilde{\mathcal{A}}$ of $\mathcal{A}$, it holds that $S^{\tilde{\mathcal{A}}, \tilde{\mathcal{A}}^{-1}}$ P -breaks U .

The following result from [12] shows the relation between BB constructions and oracle separations.

Theorem 3 (Theorem 4.2 in [12]). Assume there exists a BB construction of a primitive Q from a primitive P with inverse access. Then, for any unitary $\mathcal{O}$, if P exists relative to $(\mathcal{O}, \mathcal{O}^{-1})$, then Q exists relative to $(\mathcal{O}, \mathcal{O}^{-1})$.

3.5 Statistical Short Pseudorandom States

We include some results that will be used in our separation. Intuitively, these results states that a sufficiently long random string enables the construction of a sufficiently small PRS. Notice that this does not imply that SPRS exist unconditionally, since a generator for a SPRS is not allowed to use an arbitrarily long input to generate the state.

Theorem 4 (Theorem 5.1 in [10]). There exists a generator Gen such that for every $n \in \mathbb{N}$ number of qubits, $5 \leq \lambda \in \mathbb{N}$ security parameter and $t \in \mathbb{N}$ number of copies, satisfies the following trace distance bound:

$$\text{Tr}(\mathcal{D}_1, \mathcal{D}_2) \leq (t + 8) \cdot e^{-\lambda} + (5\sqrt{t} + \lambda + 1) \cdot 2^{-\lambda} + 2 \cdot \left(\frac{8}{10}\right)^\lambda,$$

where the distributions $\mathcal{D}_1, \mathcal{D}_2$ are defined as follows:

- $\mathcal{D}_1$: Sample $\tilde{f} \leftarrow (\{0, 1\}^{p(n, \lambda)})^{\{0, 1\}^n}$ for some fixed polynomial p , execute t times the generation algorithm $\text{Gen}^{U_{\tilde{f}}}(1^n, 1^\lambda)$, where $U_{\tilde{f}}$ is the unitarization of $\tilde{f}$, and output the t output states.
- $\mathcal{D}_2$: Sample $|\psi\rangle$ a random n -qubit state and output $|\psi\rangle^{\otimes t}$.

Corollary 2. Let $5 \leq \lambda \in \mathbb{N}$ be the security parameter and let $n = O(\log(\lambda))$. There exists a function $q = q(2^n, \lambda)$ (polynomial in λ) and a QPT algorithm G such that for any (computationally unbounded) adversary $\mathcal{A}$ and any polynomial $t = t(\lambda)$,

$$\left| \Pr_{k \leftarrow \{0, 1\}^q} \left[\mathcal{A} \left(G(k)^{\otimes t(\lambda)} \right) = 1 \right] - \Pr_{|\phi\rangle \leftarrow \text{Haar}(\mathbb{C}^n)} \left[\mathcal{A} \left(|\phi\rangle^{\otimes t(\lambda)} \right) = 1 \right] \right| \leq \text{negl}(\lambda).$$

Proof. We define $G(k)$ as follows. It chooses a random function $\tilde{f}_k : \{0, 1\}^n \rightarrow \{0, 1\}^{p(n, \lambda)}$ based on the string k . Note that as long as q is set to be a large enough

polynomial, since k is picked at random from $\{0, 1\}^q$, k contains sufficient randomness to choose a function $\tilde{f}_k$ that is uniformly random from $(\{0, 1\}^{p(n, \lambda)})^{\{0, 1\}^n}$ given that $n = O(\log(\lambda))$.

Next, $G(k)$ runs $\text{Gen}^{U_{\tilde{f}_k}}(1^n, 1^\lambda)$ and outputs the resulting state. By applying Theorem 4, we directly obtain that for any (computationally unbounded) adversary $\mathcal{A}$ and any polynomial $t = t(\lambda)$,

$$\left| \Pr_{k \leftarrow \{0, 1\}^q} \left[\mathcal{A} \left(G(k)^{\otimes t(\lambda)} \right) = 1 \right] - \Pr_{|\phi\rangle \leftarrow \text{Haar}(\mathbb{C}^n)} \left[\mathcal{A} \left(|\phi\rangle^{\otimes t(\lambda)} \right) = 1 \right] \right| \leq \text{negl}(\lambda).$$

□

4 Separation

In this section, we present our separation of PRG from PRS.

Theorem 5. *Let $\ell = \ell(\lambda) \in O(\lambda)$ be a function on the security parameter $\lambda \in \mathbb{N}$ and let $n = n(\lambda)$ be a polynomial satisfying $n > \lambda$. There does not exist a BB construction of a n -PRG from a ℓ -PRS with inverse access.*

Proof. Our approach is to show that there exists a unitary quantum oracle with inverse access, relative to which there is no BB construction of PRG from PRS with inverse access. Then, by Theorem 3, this means there cannot exist such a construction in the plain model as well.

We only show the proof for the case $n := 2\lambda + r$, where the polynomial $r = r(\lambda) := \lambda^2 \cdot q(2^{\ell(\log(\lambda))}, \lambda)$ and q is specified in Corollary 2. Specifically, r is chosen so that a randomly sampled string k of length r is sufficient to describe a set of states $\{|\phi_{x,k}\rangle\}_{x \in \{0, 1\}^d, d \leq \log(\lambda)}$, where $|\phi_{x,k}\rangle$ is of size $\ell(|x|)$, such that these states act as statistical PRS.

Note that ruling out a $(2\lambda + r)$ -PRG is sufficient to rule out all possible lengths $n > \lambda$, since any PRG can be composed sufficiently many times to build a PRG with longer output length, meaning that different output length regimes are essentially equivalent.

Our proof is relative to two unitary oracles: a (unitarized) PSPACE oracle $\mathcal{C}$ and a ℓ -CHFS oracle $\mathcal{O}$ sampled from a set of oracles $\mathcal{O}_\ell$. Given that both oracles are self-inverse, it is sufficient to give access to $\mathcal{T} := (\mathcal{C}, \mathcal{O})$.

Assume for contradiction that there exists a BB construction of a n -PRG from a ℓ -PRS. First, we state the following result which follows directly from Theorem 4.1 in [7].

Claim 1. *There exists a ℓ -PRS relative to $\mathcal{T}$. The PRS construction only uses oracle access to $\mathcal{O}$ and satisfies security with probability 1 over the distribution of $\mathcal{O}$ and satisfies correctness for all $\mathcal{O} \in \mathcal{O}_\ell$.*

By Claim 1 and the existence of a BB construction, there exists a PRG $\overline{G} : \{0, 1\}^\lambda \rightarrow \{0, 1\}^n$ relative to $\mathcal{T}$. Furthermore, $\overline{G}$ only uses oracle access to $\mathcal{O}$ and satisfies security with probability 1 over the distribution of $\mathcal{O}$ and satisfies

correctness for all $\mathcal{O}$, given that these properties are satisfied for the underlying PRS. Assume $\overline{G}$ queries the oracle at most $T = T(\lambda)$ times for some polynomial T and define the polynomial $m = m(\lambda) := \pi \cdot T \cdot \lambda$.

We now construct an algorithm $G : \{0, 1\}^{n-\lambda} \rightarrow \{0, 1\}^n$ (see Fig. 1) using $\overline{G}$, that does not query the oracle on any input of length less than $\log(\lambda)$.

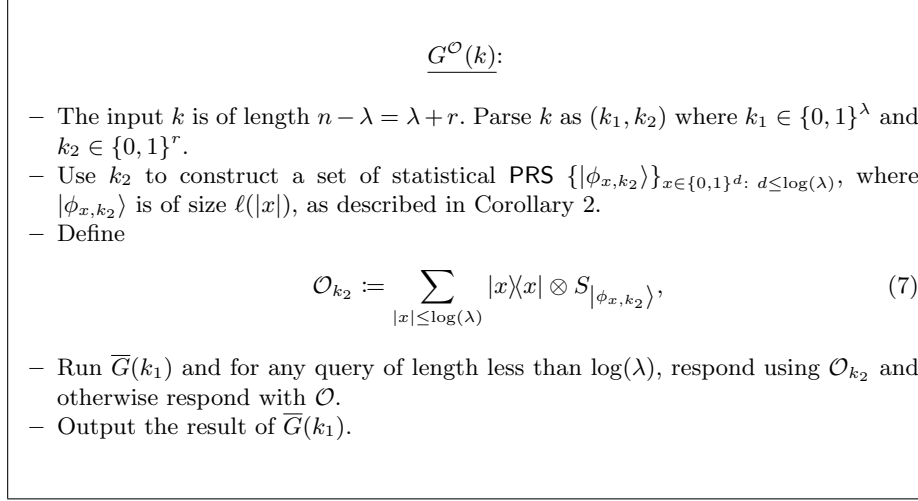


Fig. 1. Algorithm of $G^\mathcal{O}$.

By abuse of notation, we view the input length of G as λ from now on. Recall that $\overline{G}^\mathcal{O}$ satisfies correctness for any oracle $\mathcal{O} \in \mathcal{O}_\ell$. It is clear that G inherits this property. In particular, for any $\mathcal{O} \in \mathcal{O}_\ell$, there exists a negligible function ϵ (may depend on $\mathcal{O}$) such that: for all $\lambda \in \mathbb{N}$ and for any $k \leftarrow \{0, 1\}^\lambda$, there exists a string y_k such that

$$\Pr_{k \leftarrow \{0, 1\}^\lambda} [G^\mathcal{O}(k) = y_k] \geq 1 - \epsilon(\lambda). \quad (8)$$

Definition 10. For any pair of oracles $\mathcal{O}, \hat{\mathcal{O}} \in \mathcal{O}_\ell$ and $\lambda \in \mathbb{N}$, define the set $Disagr_\lambda^{\mathcal{O}, \hat{\mathcal{O}}}$ as the set of inputs $k \in \{0, 1\}^\lambda$ such that,

$$\Pr [G^\mathcal{O}(k) \neq G^{\hat{\mathcal{O}}}(k)] \geq 1/3. \quad (9)$$

Lemma 2. There exists some integer $\lambda^* > 0$, such that any pair of oracles $\mathcal{O}, \hat{\mathcal{O}} \in \mathcal{O}_\ell$ satisfy the following: for all integers $\lambda > \lambda^*$,

$$\Pr_{k \leftarrow \{0, 1\}^\lambda} [k \in Disagr_\lambda^{\mathcal{O}, \hat{\mathcal{O}}}] < \frac{1}{\lambda}.$$

Proof. Assume for contradiction that the claim does not hold. This means that there exists an infinite sequence $(\lambda_j)_{j \in \mathbb{N}}$ of integers such that for each $j \in \mathbb{N}$ there exists a pair of oracle $\mathcal{O}^j, \hat{\mathcal{O}}^j \in \mathcal{O}_\ell$ satisfying:

$$\Pr_{k \leftarrow \{0,1\}^{\lambda_j}} \left[k \in \text{Disagr}_{\lambda_j}^{\mathcal{O}^j, \hat{\mathcal{O}}^j} \right] \geq \frac{1}{\lambda_j}.$$

Fix $j \in \mathbb{N}$. We can define a sequence of m oracles $\mathcal{O}_1^j, \dots, \mathcal{O}_m^j$ starting with $\mathcal{O}_1^j = \mathcal{O}^j$ and ending with $\mathcal{O}_m^j = \hat{\mathcal{O}}^j$ such that for any $i \in [m]$ and input ρ ,

$$\text{Tr} \left(\mathcal{O}_i^j(\rho), \mathcal{O}_{i+1}^j(\rho) \right) \leq \frac{\pi}{m}. \quad (10)$$

It is not difficult to see that such a sequence must exist. In particular, for any $i \in [m]$, let Φ_i^j be the product state that classifies the oracle $\mathcal{O}_i^j$ as described in Definition 6. To build this sequence, we gradually “move” all the output states of Φ_1^j toward that of Φ_m^j , reducing the trace distance by at most π/m at each step. After at most m steps, we reach Φ_m^j . For this argument, note that the longest geodesic distance between any two states in their spherical representation is at most π and such a path can be visualized as a semi-circle. If this path is split into m equally spaced states, it is easy to show that the trace distance between any two consecutive states in this sequence is at most π/m .

Given that G queries the oracle at most T times, it cannot distinguish oracle access to $\mathcal{O}_i^j$ or $\mathcal{O}_{i+1}^j$ with better than $\frac{T}{m} < \frac{1}{\lambda_j}$ probability. Therefore, for any input $k \in \{0,1\}^{\lambda_j}$,

$$\text{Tr} \left(G^{\mathcal{O}_i^j}(k), G^{\mathcal{O}_{i+1}^j}(k) \right) \leq \frac{1}{\lambda_j}. \quad (11)$$

Now recall that there exists a set of inputs $\text{Disagr}_{\lambda_j}^{\mathcal{O}^j, \hat{\mathcal{O}}^j}$ where $G^{\mathcal{O}^j}$ and $G^{\hat{\mathcal{O}}^j}$ do not agree with at least $1/3$ probability.

For any input $k \in \text{Disagr}_{\lambda_j}^{\mathcal{O}^j, \hat{\mathcal{O}}^j}$ there are three cases that can occur:

1. There exists distinct values $y_k, \hat{y}_k$ such that

$$\Pr \left[G^{\mathcal{O}^j}(k) = y_k \right] \geq \frac{9}{10} \quad (12)$$

$$\Pr \left[G^{\hat{\mathcal{O}}^j}(k) = \hat{y}_k \right] \geq \frac{9}{10}. \quad (13)$$

2. There does not exist a value y_k such that Eq. (12) holds. This implies that $G^{\mathcal{O}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic i.e. evaluating $G^{\mathcal{O}^j}(k)$ twice yields two distinct values with at least $\frac{1}{10}$ probability.
3. There does not exist a value $\hat{y}_k$ such that Eq. (13) holds. This implies that $G^{\hat{\mathcal{O}}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic.

We analyze each case separately.

– (Case 1): Consider the following probabilities:

$$p_{k,i} := \Pr \left[G^{\mathcal{O}_i^j}(k) = y_k \right] \quad (14)$$

Notice that, by Eqs. (12) and (13), we have $p_{k,1} \geq 9/10$ and $p_{k,m} \leq 1/10$. On the other hand, Eq. (11) states the output distribution under two consecutive oracles differs by at most $\frac{1}{\lambda_j}$. This means that there exists $i_k \in [m]$ such that

$$\frac{1}{4} \leq \frac{1}{2} - \frac{2}{\lambda_j} \leq \Pr \left[G^{\mathcal{O}_{i_k}^j}(k) = y_k \right] \leq \frac{1}{2} + \frac{2}{\lambda_j} \leq \frac{3}{4}. \quad (15)$$

In other words, $G^{\mathcal{O}_{i_k}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic.

- (Case 2): In this case, $G^{\mathcal{O}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic so set i_k to 1, noting that $\mathcal{O}^j = \mathcal{O}_1^j$.
- (Case 3): In this case, $G^{\hat{\mathcal{O}}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic so set i_k to m , noting that $\hat{\mathcal{O}}^j = \mathcal{O}_m^j$.

In other words, in all cases, there is an index i_k such that $G^{\mathcal{O}_{i_k}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic.

More generally, for every input in $\text{Disagr}_{\lambda_j}^{\mathcal{O}^j, \hat{\mathcal{O}}^j}$ there exists an oracle in the sequence under which the evaluation of G is $(\frac{1}{10})$ -non-deterministic.

The number of inputs in the set $\text{Disagr}_{\lambda_j}^{\mathcal{O}^j, \hat{\mathcal{O}}^j}$ is at least $\frac{2^{\lambda_j}}{\lambda_j}$. Therefore, there is an index, which we denote by i_j , such that there are at least $\frac{2^{\lambda_j}}{m\lambda_j}$ inputs $k \in \{0, 1\}^{\lambda_j}$, where $G^{\mathcal{O}_{i_j}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic.

By performing this analysis for each $j \in \mathbb{N}$, we establish that there is some oracle $\bar{\mathcal{O}}^j$ such that $G^{\bar{\mathcal{O}}^j}$ is $(\frac{1}{10})$ -non-deterministic on at least $\frac{1}{m\lambda_j}$ fraction of inputs $k \in \{0, 1\}^{\lambda_j}$. Let $\bar{\Phi}^j := \left\{ |\bar{\phi}_x^j\rangle \right\}_{x \in \{0, 1\}^{\lambda_j}}$ be the product state classifying the oracle $\bar{\mathcal{O}}^j$.

Assume without loss of generality, that for every $j \in \mathbb{N}$, $\lambda_{j+1} > 2^{2^{\lambda_j}}$. If the sequence does not satisfy this requirement, we can simply take a subsequence which does.

Notice that for any input $k_j \in \{0, 1\}^{\lambda_j}$, $G^{\mathcal{O}}(k_j)$ does not query the oracle on input of length less than $\log(\lambda_j)$ by definition, and since G is a QPT algorithm, it does not query on an input of length larger than 2^{λ_j} . Similarly, for any input $k_{j+1} \in \{0, 1\}^{\lambda_{j+1}}$, $G^{\mathcal{O}}(k_{j+1})$ does not query the oracle on input of length less than $\log(\lambda_{j+1}) > \log(2^{2^{\lambda_j}}) = 2^{\lambda_j}$, which is larger than the maximum query length when run on k_j . In other words, the queries of G do not intersect under different security parameters in the sequence $(\lambda_j)_{j \in \mathbb{N}}$.

This allows us to define the following product state:

$$\bar{\Phi} := \left\{ |\bar{\phi}_x^j\rangle |1\rangle \right\}_{x \in U_j} \cup \left\{ |0^{\ell(|x|)}\rangle |1\rangle \right\}_{x \in U'} . \quad (16)$$

Here, $U_j := \{x : x \in \{0, 1\}^* \wedge \log(\lambda_j) \leq |x| \leq 2^{\lambda_j}\}$ and $U' := \{x : \forall j \in \mathbb{N}, x \notin U_j\}$.

Define the oracle $\overline{\mathcal{O}}$ as the one determined by $\overline{\Phi}$ as described in Definition 6. Notice that for an input ρ of size $\log(\lambda_j) \leq |\rho| \leq 2^{\lambda_j}$, we have $\overline{\mathcal{O}}(\rho) = \overline{\mathcal{O}}^j(\rho)$.

To summarize, for every $j \in \mathbb{N}$ we have

- $G^{\overline{\mathcal{O}}^j}$ is $(\frac{1}{10})$ -non-deterministic on $\frac{1}{m\lambda_j}$ fraction of inputs in $\{0, 1\}^{\lambda_j}$.
- For an input $k_j \in \{0, 1\}^{\lambda_j}$, $G^{\overline{\mathcal{O}}^j}(k_j)$ queries the oracle on inputs of length at most 2^{λ_j} and at least $\log(\lambda_j)$.
- For any state ρ of size $\log(\lambda_j) \leq |\rho| \leq 2^{\lambda_j}$, $\overline{\mathcal{O}}(\rho) = \overline{\mathcal{O}}^j(\rho)$.

Therefore, there is an infinite sequence $(\lambda_j)_{j \in \mathbb{N}}$ of security parameters such that, for each $j \in \mathbb{N}$, $G^{\overline{\mathcal{O}}}$ is $(\frac{1}{10})$ -non-deterministic on an inverse-polynomial fraction $(\frac{1}{\lambda_j^m})$ of inputs in $\{0, 1\}^{\lambda_j}$. This contradicts the correctness requirement for G (as described in Eq. (8)). $\square$

Define the oracle I to be the element of $\mathcal{O}_\ell$ that is the identity mapping. Define the oracle $\mathcal{T}_{\geq w} := (\mathcal{C}, \mathcal{O}_{\geq w})$ where $\mathcal{O}_{\geq w}$ only answers queries of length at least w .

Consider the following hybrids of PRG security experiment.

- $\text{Exp}_1^A(\lambda)$:
 1. Sample oracle $\mathcal{O} \leftarrow \mathcal{O}_\ell$.
 2. Sample $k \leftarrow \{0, 1\}^\lambda$ and $b \leftarrow \{0, 1\}$.
 3. If $b = 0$, generate $y \leftarrow \overline{G}^{\mathcal{O}}(k)$. Else, sample $y \leftarrow \{0, 1\}^n$.
 4. $b' \leftarrow \mathcal{A}^T(y)$.
 5. If $b' = b$, output 1. Otherwise, output 0.
- $\text{Exp}_2^A(\lambda)$:
 1. Sample oracle $\mathcal{O} \leftarrow \mathcal{O}_\ell$.
 2. Sample $k \leftarrow \{0, 1\}^\lambda$ and $b \leftarrow \{0, 1\}$.
 3. If $b = 0$, generate $y \leftarrow \overline{G}^{\mathcal{O}}(k)$. Else, sample $y \leftarrow \{0, 1\}^n$.
 4. $b' \leftarrow \mathcal{A}^{\mathcal{T}_{\geq \log(\lambda)}}(y)$. Notice that $\mathcal{A}$ only has access to $\mathcal{T}_{\geq \log(\lambda)}$ in this variant.
 5. If $b' = b$, output 1. Otherwise, output 0.
- $\text{Exp}_3^A(\lambda)$:
 1. Sample oracle $\mathcal{O} \leftarrow \mathcal{O}_\ell$.
 2. Sample $k \leftarrow \{0, 1\}^{n-\lambda}$ and $b \leftarrow \{0, 1\}$.
 3. If $b = 0$, generate $y \leftarrow G^{\mathcal{O}}(k)$. Else, sample $y \leftarrow \{0, 1\}^n$.
 4. $b' \leftarrow \mathcal{A}^{\mathcal{T}_{\geq \log(\lambda)}}(y)$.
 5. If $b' = b$, output 1. Otherwise, output 0.
- $\text{Exp}_4^A(\lambda)$:
 1. Sample oracle $\mathcal{O} \leftarrow \mathcal{O}_\ell$.
 2. Sample $k \leftarrow \{0, 1\}^{n-\lambda}$ and $b \leftarrow \{0, 1\}$.
 3. If $b = 0$, generate $y \leftarrow G^I(k)$. Else, sample $y \leftarrow \{0, 1\}^n$.
 4. $b' \leftarrow \mathcal{A}^{\mathcal{T}_{\geq \log(\lambda)}}(y)$.

5. If $b' = b$, output 1. Otherwise, output 0.
- $\text{Exp}_5^A(\lambda)$:
1. Sample oracle $\mathcal{O} \leftarrow \mathcal{O}_\ell$.
 2. Sample $k \leftarrow \{0, 1\}^{n-\lambda}$ and $b \leftarrow \{0, 1\}$.
 3. If $b = 0$, generate $y \leftarrow G^{\mathcal{O}}(k)$. Else, sample $y \leftarrow \{0, 1\}^n$.
 4. $b' \leftarrow \mathcal{A}^{\mathcal{C}}(y)$.
 5. If $b' = b$, output 1. Otherwise, output 0.

It was shown in [17] using a variant of Borel-Cantelli Lemma (Lemma 2.9 in [17]) that if a primitive A exists in an idealized model and there is an adversary against another primitive B with constant advantage, say $1/10$, (over the distribution of oracles) then this is sufficient to rule out BB constructions of B from A . More formally, Lemma 2.9 in [17] gives the following result.

Claim 2. *For any QPT adversary $\mathcal{A}$ and large enough λ ,*

$$\Pr \left[\text{Exp}_1^A(\lambda) = 1 \right] \leq \frac{1}{2} + \frac{1}{10}.$$

We now show that the experiment hybrids are indistinguishable except with small probability.

Claim 3. *For any QPT algorithm $\mathcal{A}$, there exists a QPT algorithm $\mathcal{B}$ such that*

$$\Pr \left[\text{Exp}_2^A(\lambda) = 1 \right] \leq \Pr \left[\text{Exp}_1^B(\lambda) = 1 \right]. \quad (17)$$

Proof. This is clear because the only difference between these experiments is that the adversary's oracle access is restricted. $\square$

Claim 4. *For any QPT algorithm $\mathcal{A}$, there exists a QPT algorithm $\mathcal{B}$ such that,*

$$\Pr \left[\text{Exp}_3^B(\lambda) = 1 \right] \leq \Pr \left[\text{Exp}_2^A(\lambda) = 1 \right] + \text{negl}(\lambda). \quad (18)$$

Proof. Notice that the only real difference between these hybrids is that in $\text{Exp}_2^A(\lambda)$, the evaluation $\overline{G}^{\mathcal{O}}(k)$ uses $\mathcal{O}$ for all queries. Meanwhile, in $\text{Exp}_3^B(\lambda)$, the evaluation of $G^{\mathcal{O}}(k)$ runs $\overline{G}$ but with oracle access to $\mathcal{O}$ restricted only on queries of length larger than $\log(\lambda)$, and for shorter queries, it uses an oracle simulated using the randomness of the input (see Fig. 1).

By Corollary 2, the states generated using the random input k_2 (see Fig. 1) are statistically indistinguishable from Haar random states. As a result, the simulated oracle $\mathcal{O}_{k_2}$ is statistically indistinguishable from a CHFS oracle. Hence, if the results of the experiments can be distinguished, this can be converted into an attack against Corollary 2. $\square$

Claim 5. *For any QPT algorithm $\mathcal{A}$ and large enough λ ,*

$$\Pr \left[\text{Exp}_4^A(\lambda) = 1 \right] \leq \Pr \left[\text{Exp}_3^A(\lambda) = 1 \right] + \frac{1}{3} + \frac{1}{\lambda}. \quad (19)$$

Proof. The only difference between these hybrids is G is given access to I in $\text{Exp}_4^A(\lambda)$ and is given access to $\mathcal{O}$ in $\text{Exp}_3^A(\lambda)$.

By Lemma 2, there exists a constant λ^* such that for any $\lambda > \lambda^*$, we have

$$\Pr_{k \leftarrow \{0,1\}^\lambda} \left[k \in \text{Disagr}_\lambda^{I, \mathcal{O}} \right] < \frac{1}{\lambda}.$$

Therefore, there is at least $1 - \frac{1}{\lambda}$ probability that the key k sampled in the experiment does not belong to this set. In this case, we have

$$\Pr \left[G^{\mathcal{O}}(k) = G^I(k) \right] \geq 2/3. \quad (20)$$

Therefore, $\text{Exp}_3^A(\lambda)$ and $\text{Exp}_4^A(\lambda)$ can only be distinguished with at most

$$\left(1 - \frac{1}{\lambda} \right) \cdot \frac{1}{3} + \frac{1}{\lambda} \cdot 1 < \frac{1}{3} + \frac{1}{\lambda}$$

probability. $\square$

Claim 6. *For any QPT algorithm $\mathcal{A}$, there exists a QPT algorithm $\mathcal{B}$ such that*

$$\Pr \left[\text{Exp}_5^A(\lambda) = 1 \right] \leq \Pr \left[\text{Exp}_4^B(\lambda) = 1 \right]. \quad (21)$$

Proof. This is clear because the only difference between these experiments is that the adversary's oracle access is restricted. $\square$

As a result of Claims 2, 3, 4, 5, and 6, and the triangle inequality, we have for large enough λ ,

$$\Pr \left[\text{Exp}_5^A(\lambda) = 1 \right] \leq \frac{1}{2} + \frac{1}{10} + \frac{1}{3} + \frac{1}{\lambda} + \text{negl}(\lambda) = \frac{14}{15} + \frac{1}{\lambda} + \text{negl}(\lambda). \quad (22)$$

Notice that $\text{Exp}_5^A(\lambda)$ is just the PRG security experiment for G^I against $\mathcal{A}^c$. However, G^I can be easily simulated with a QPT algorithm without any oracle access since I is just the identity.

On the other hand, there exists a trivial search attack, using a PSPACE oracle, against PRG security, given that any polynomial-space quantum computations with classical inputs can be simulated using a PSPACE oracle. In particular, there exists an adversary $\bar{\mathcal{A}}$ such that

$$\Pr \left[\text{Exp}_5^{\bar{\mathcal{A}}}(\lambda) = 1 \right] \geq 1 - \text{negl}(\lambda).$$

contradicting Eq. (22) above.

Therefore, there does not exist a BB construction of a n -PRG from a ℓ -PRS. $\square$

5 Separating PRG from $\perp$ -PRG

To demonstrate the separation between PRGs and $\perp$ -PRGs, we will use two independent oracles: an oracle for a PSPACE-complete language $\mathcal{C}$ and a $\perp$ -pseudodeterministic random oracle $\mathcal{O}$ described in Construction 1, similar to one used in [3].

Construction 1. Let $\lambda \in \mathbb{N}$ be the security parameter. Let $c > 0$ be a constant. Fix a pseudodeterminism error $\mu(\lambda) = \lambda^{-c}$. Let ℓ be polynomial such that $\ell(\lambda) > \lambda$. Sample random functions $Q_\lambda \leftarrow \Pi_{\lambda,\lambda}$ and $O_\lambda \leftarrow \Pi_{\lambda,\ell}$. Sample a random set $\mathcal{P}_\lambda \subset \{0,1\}^\lambda$ of size $\lfloor \mu \cdot 2^\lambda \rfloor$ and let P_λ be the function that outputs 1 on input $x \in \mathcal{P}_\lambda$ and 0 otherwise. The oracle $\mathcal{O}_\lambda := \mathcal{O}[P_\lambda, Q_\lambda, O_\lambda]$ is a unitary version of the algorithm that acts as follows on any $x \in \{0,1\}^\lambda$:

- Compute $y = O_\lambda(x)$.
- Compute $q = Q_\lambda(x)$ and let $p_x := q/2^\lambda$, where q is interpreted as an integer in $[0 : 2^\lambda]$.
- If $P_\lambda(x) = 1$ set $|\phi_x\rangle := \sqrt{p_x}|\perp\rangle + \sqrt{1-p_x}|y\rangle$.
- Otherwise, let $|\phi_x\rangle := |y\rangle$.
- Measure $|\phi_x\rangle$ in the computational basis and output the result.

We define the “good” set $\mathcal{G}_\lambda^\mathcal{O}$ for $\mathcal{O}_\lambda$ as follows:

$$\mathcal{G}_\lambda^\mathcal{O} := \{x \in \{0,1\}^\lambda : P_\lambda(x) = 0\}.$$

Furthermore, we let $\mathcal{O}_{\mu,\ell}$ denote the set of all oracles described in Construction 1.

The following lemma follows directly from the definition of $\mathcal{O}$.

Lemma 3. $\mathcal{O}_\lambda$ has the following properties:

- $\Pr_{x \leftarrow \{0,1\}^\lambda} [x \in \mathcal{G}_\lambda^\mathcal{O}] \geq 1 - \mu$.
- For every $x \in \mathcal{G}_\lambda^\mathcal{O}$, there exists a non- $\perp$ value $y \in \{0,1\}^\ell$ such that:

$$\Pr[\mathcal{O}_\lambda(x) = y] = 1.$$

- For every $x \notin \mathcal{G}_\lambda^\mathcal{O}$, there exists a probability $p_x \in [0,1]$ and non- $\perp$ value $y \in \{0,1\}^\ell$ such that:
 1. $\Pr[y \leftarrow \mathcal{O}_\lambda(x)] = 1 - p_x$.
 2. $\Pr[\perp \leftarrow \mathcal{O}_\lambda(x)] = p_x$.

Theorem 6. Let $\lambda \in \mathbb{N}$ be the security parameter. For any polynomials $w(\lambda), \ell(\lambda) > \lambda$ and pseudodeterminism error $\mu(\lambda) = O(\lambda^{-c})$ for $c > 0$, there does not exist a BB construction of a w -PRG from a $(3\mu, \ell)$ - $\perp$ -PRG with inverse access.

Proof. Our approach is to show that relative to the self-inverse unitary quantum oracles $\mathcal{T} := (\mathcal{C}, \mathcal{O})$, there is no BB construction of PRG from $\perp$ -PRG. Then, by Theorem 3, this means there cannot exist such a construction in the plain model as well.

We only show the proof for the case $w := 2\lambda + r$, where $r = r(\lambda) := 16\lambda^3$. Specifically, r is chosen so that a randomly sampled string k of length r is sufficient to describe sampling a set of oracles $(\mathcal{O}_n)_{n \leq \log(\lambda)}$ in the same way as in Construction 1.

Note that ruling out a $(2\lambda + r)$ -PRG is sufficient to rule out all possible lengths $w > \lambda$, since any PRG can be composed sufficiently many times to build a PRG with longer output length, meaning that different output length regimes are essentially equivalent.

Assume for contradiction that there exists a BB construction of a PRG $\overline{G}^F$ from a $(3\mu, \ell)$ - $\perp$ -PRG F . First, we show that there exists a (μ, ℓ) - $\perp$ -PRG relative to the oracles $(\mathcal{O}, \mathcal{C})$.

Claim 7. *Under security parameter $\lambda \in \mathbb{N}$, the sequence of functions $\{\mathcal{O}_\lambda[P_\lambda, Q_\lambda, O_\lambda]\}_{\lambda \in \mathbb{N}}$ is a $(\mu(\lambda), \ell(\lambda))$ - $\perp$ -PRG for all possible sequences P and Q and with probability 1 over the distribution of O . Furthermore, correctness is satisfied for all possible oracles.*

Proof. This follows from Claim 10 in [4], but we give the proof again here for completeness.

By Lemma 3, $\mathcal{O}$ satisfies the correctness/pseudodeterminism condition of a (μ, ℓ) - $\perp$ -PRG.

For security, we need to show that for any P, Q and with probability 1 over the distribution of O : for every non-uniform QPT distinguisher $\mathcal{A}$ and polynomial $q = q(\lambda)$:

$$\left| \Pr \left[\begin{array}{l} k \leftarrow \{0, 1\}^\lambda \\ y_1 \leftarrow \mathcal{O}_\lambda(k) \\ \vdots \\ y_q \leftarrow \mathcal{O}_\lambda(k) \end{array} : \mathcal{A}^{\mathcal{O}, \mathcal{C}}(y_1, \dots, y_q) = 1 \right] - \Pr \left[\begin{array}{l} k \leftarrow \{0, 1\}^\lambda \\ y \leftarrow \{0, 1\}^\ell \\ y_1 \leftarrow \text{ls-}\perp(\mathcal{O}_\lambda(k), y) \\ \vdots \\ y_q \leftarrow \text{ls-}\perp(\mathcal{O}_\lambda(k), y) \end{array} : \mathcal{A}^{\mathcal{O}, \mathcal{C}}(y_1, \dots, y_q) = 1 \right] \right| \leq \text{negl}(\lambda)$$

Let Z_λ be the function that outputs 0^ℓ on any input and let $\mathcal{Z}_\lambda := \mathcal{O}[P_\lambda, Q_\lambda, Z_\lambda]$. Note that

- $\mathcal{Z}_\lambda$ is independent of O_λ ,
- $\mathcal{O}_\lambda(k) = \text{ls-}\perp(\mathcal{O}_\lambda(k), O_\lambda(k)) = \text{ls-}\perp(\mathcal{Z}_\lambda(k), O_\lambda(k))$,
- $\text{ls-}\perp(\mathcal{O}_\lambda(k), y) = \text{ls-}\perp(\mathcal{Z}_\lambda(k), y)$.

Therefore, $\mathcal{A}^{\mathcal{O}, \mathcal{C}}$ needs to distinguish between evaluations of $\text{ls-}\perp(\mathcal{Z}_\lambda(k), y)$ and $\text{ls-}\perp(\mathcal{Z}_\lambda(k), O_\lambda(k))$.

Lemma 2.2 from [21] states that a random oracle acts as a PRG i.e.:

$$\mathbb{E}_{O \leftarrow \Pi_{\lambda, \ell}} \left[\left| \Pr_{k \leftarrow \{0, 1\}^\lambda} [\mathcal{A}^O(O(k)) = 1] - \Pr_{y \leftarrow \{0, 1\}^\ell} [\mathcal{A}^O(y) = 1] \right| \right] \leq \frac{1}{2^{\lambda/4}}.$$

Note that this result even holds against unbounded-time adversaries as long as the number of queries to the oracle is polynomial. Hence, this result also holds

against adversaries with access to a PSPACE-oracle:

$$\mathbb{E}_{O \leftarrow \Pi_{\lambda, \ell}} \left[\left| \Pr_{k \leftarrow \{0,1\}^\lambda} [\mathcal{A}^{O, \mathcal{C}}(O(k)) = 1] - \Pr_{y \leftarrow \{0,1\}^\ell} [\mathcal{A}^{O, \mathcal{C}}(y) = 1] \right| \right] \leq \frac{1}{2^{\lambda/4}}.$$

Next, notice that for any functions P, Q , distinguishing between evaluations of $\text{ls-}\perp(\mathcal{Z}_\lambda(k), y)$ and $\text{ls-}\perp(\mathcal{Z}_\lambda(k), O_\lambda(k))$ is just as hard as distinguishing the two scenarios in the equation above, given that $\mathcal{Z}_\lambda$ is independent of O_λ . Therefore,

$$\mathbb{E}_{O \leftarrow \Pi_{\lambda, \ell}} \left[\left| \Pr_{(y_1, \dots, y_q) \leftarrow D_{\mathcal{Z}, O}^0} [\mathcal{A}^{O, \mathcal{C}}(y_1, \dots, y_q) = 1] - \Pr_{(y_1, \dots, y_q) \leftarrow D_{\mathcal{Z}}^1} [\mathcal{A}^{O, \mathcal{C}}(y_1, \dots, y_q) = 1] \right| \right] \leq 2^{-\lambda/4}$$

where,

$$D_{\mathcal{Z}, O}^0 := \begin{bmatrix} k \leftarrow \{0,1\}^\lambda \\ y_1 \leftarrow \text{ls-}\perp(\mathcal{Z}_\lambda(k), O_\lambda(k)) \\ \vdots \\ y_q \leftarrow \text{ls-}\perp(\mathcal{Z}_\lambda(k), O_\lambda(k)) \end{bmatrix} \quad D_{\mathcal{Z}}^1 := \begin{bmatrix} k \leftarrow \{0,1\}^\lambda \\ y \leftarrow \{0,1\}^\ell \\ y_1 \leftarrow \text{ls-}\perp(\mathcal{Z}_\lambda(k), y) \\ \vdots \\ y_q \leftarrow \text{ls-}\perp(\mathcal{Z}_\lambda(k), y) \end{bmatrix}$$

By Markov inequality, we get that

$$\Pr_{O \leftarrow \Pi_{\lambda, \ell}} \left[\left| \Pr_{(y_1, \dots, y_q) \leftarrow D_{\mathcal{Z}, O}^0} [\mathcal{A}^{O, \mathcal{C}}(y_1, \dots, y_q) = 1] - \Pr_{(y_1, \dots, y_q) \leftarrow D_{\mathcal{Z}}^1} [\mathcal{A}^{O, \mathcal{C}}(y_1, \dots, y_q) = 1] \right| \geq 2^{-\lambda/8} \right] \leq 2^{-\lambda/8}$$

By Borel-Cantelli Lemma, since $\sum_\lambda 2^{-\lambda/8}$ converges, with probability 1 over the distribution of O , it holds that

$$\left| \Pr_{(y_1, \dots, y_q) \leftarrow D_{\mathcal{Z}, O}^0} [\mathcal{A}^{O, \mathcal{C}}(y_1, \dots, y_q) = 1] - \Pr_{(y_1, \dots, y_q) \leftarrow D_{\mathcal{Z}}^1} [\mathcal{A}^{O, \mathcal{C}}(y_1, \dots, y_q) = 1] \right| \leq 2^{-\lambda/8},$$

except for finitely many $\lambda \in \mathbb{N}$. There are countable number of quantum algorithms $\mathcal{A}$ making polynomial queries to $(\mathcal{O}, \mathcal{C})$, so this bound holds for every such adversary. Therefore, $\mathcal{O}$ is a $\perp$ -PRG for any P, Q and with probability 1 over the distribution of O . $\square$

Assume $\overline{G}$ queries the oracle at most T times and on inputs of length at most T , where $T = T(\lambda)$ is some polynomial on λ and define the polynomial $m = m(\lambda) := 2 \cdot \pi \cdot T \cdot \lambda$.

We now construct an algorithm $G : \{0,1\}^{w-\lambda} \rightarrow \{0,1\}^w$ (see Fig. 2) using $\overline{G}$, that does not query the oracle on any input of length less than $\log(\lambda)$.

By abuse of notation, we view the input length of G as λ from now on. Recall that $\overline{G}^{\mathcal{O}}$ satisfies correctness for any oracle $\mathcal{O} \in \mathcal{O}_{\mu, \ell}$. It is clear that G inherits this property. In particular, for any $\mathcal{O} \in \mathcal{O}_{\mu, \ell}$, there exists a negligible function

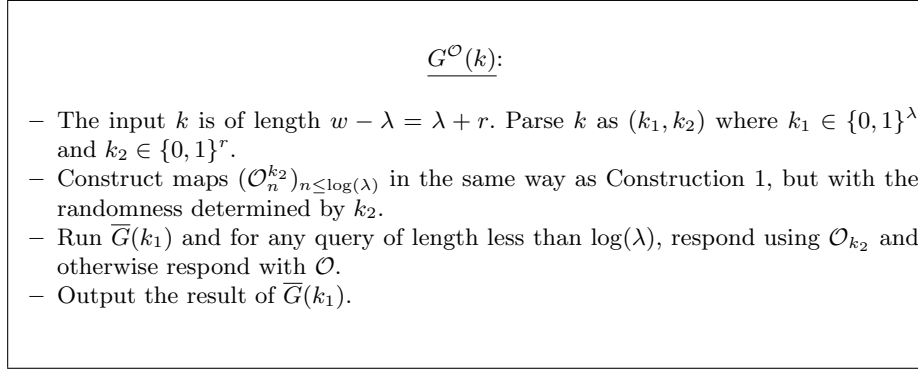


Fig. 2. Algorithm of $G^{\mathcal{O}}$.

ϵ (may depend on $\mathcal{O}$) such that: for all $\lambda \in \mathbb{N}$ and for any $k \leftarrow \{0, 1\}^\lambda$, there exists a string y_k such that

$$\Pr_{k \leftarrow \{0, 1\}^\lambda} [G^{\mathcal{O}}(k) = y_k] \geq 1 - \epsilon(\lambda). \quad (23)$$

Definition 11. For any pair of oracles $\mathcal{O}, \hat{\mathcal{O}} \in \mathcal{O}_{\mu, \ell}$ and $\lambda \in \mathbb{N}$, define the set $\text{Disagr}_\lambda^{\mathcal{O}, \hat{\mathcal{O}}}$ as the set of inputs $k \in \{0, 1\}^\lambda$ such that,

$$\Pr [G^{\mathcal{O}}(k) \neq G^{\hat{\mathcal{O}}}(k)] \geq 1/3. \quad (24)$$

Lemma 4. There exists some integer $\lambda^* > 0$, such that any pair of oracles $\mathcal{O}, \hat{\mathcal{O}} \in \mathcal{O}_{\mu, \ell}$ satisfy the following: for all integers $\lambda > \lambda^*$,

$$\Pr_{k \leftarrow \{0, 1\}^\lambda} [k \in \text{Disagr}_\lambda^{\mathcal{O}, \hat{\mathcal{O}}}] < \frac{1}{\lambda}.$$

Proof. Assume for contradiction that the claim does not hold. This means that there exists an infinite sequence $(\lambda_j)_{j \in \mathbb{N}}$ of integers such that for each $j \in \mathbb{N}$ there exists a pair of oracle $\mathcal{O}^j, \hat{\mathcal{O}}^j \in \mathcal{O}_{\mu, \ell}$ satisfying:

$$\Pr_{k \leftarrow \{0, 1\}^{\lambda_j}} [k \in \text{Disagr}_{\lambda_j}^{\mathcal{O}^j, \hat{\mathcal{O}}^j}] \geq \frac{1}{\lambda_j}.$$

Claim 8. Fix $j \in \mathbb{N}$. We can define a sequence of $n \in \text{poly}(\lambda)$ oracles $\mathcal{O}_1^j, \dots, \mathcal{O}_n^j$ starting with $\mathcal{O}_1^j = \mathcal{O}^j$ and ending with $\mathcal{O}_n^j = \hat{\mathcal{O}}^j$ such that:

1. For any $i \in [n]$ and input ρ of size $|\rho| \leq T$.

$$\text{Tr}(\mathcal{O}_i^j(\rho), \mathcal{O}_{i+1}^j(\rho)) \leq \frac{1}{m}. \quad (25)$$

2. For any $i \in [n]$, $\mathcal{O}_i^j \in \mathcal{O}_{3\mu, \ell}$.
3. $n \leq 24m/\mu$.

Proof. Define the set

$$S_{T,q} := \{x \in \{0,1\}^k : k \leq T \wedge (q-1)2^k\mu \leq x \leq q2^k\mu\} \quad (26)$$

where x is interpreted as an integer in the condition in the equation above.

Let $(O_{\lambda,i}^j, P_{\lambda,i}^j, Q_{\lambda,i}^j)_{\lambda \in \mathbb{N}}$ denote the sequence of algorithms that describe the oracle $\mathcal{O}_i^j$. Let $(O_{\lambda}^j, P_{\lambda}^j, Q_{\lambda}^j)_{\lambda \in \mathbb{N}}$ and $(\hat{O}_{\lambda}^j, \hat{P}_{\lambda}^j, \hat{Q}_{\lambda}^j)_{\lambda \in \mathbb{N}}$ denote the sequences that describe $\mathcal{O}^j$ and $\hat{\mathcal{O}}^j$, respectively. We ignore the subscript λ in the algorithms for simplicity.

To build the sequence of oracles, we gradually “move” the oracle $\mathcal{O}^j$ to $\hat{\mathcal{O}}^j$ in small steps as described in Algorithm 1 such that every step involves changing the evaluation of the oracle at most $1/m$ in trace distance.

By checking the algorithm, it can be seen that for any consecutive pair of oracles and input ρ of size $|\rho| \leq T$, the following condition is satisfied:

$$\text{Tr}(\mathcal{O}_i^j(\rho), \mathcal{O}_{i+1}^j(\rho)) \leq \frac{1}{m}. \quad (27)$$

To show the second condition, note that for any oracle $\mathcal{O}_i^j$, the portion of “bad” inputs such that $\mathcal{O}_i^j$ may return $\perp$ is bounded by the number of inputs x such that $P_i^j(x) = 1$. It can be inductively checked that this function outputs 1 on at most 3μ portion of inputs. Hence, we have that $\mathcal{O}_i^j \in \mathcal{O}_{3\mu, \ell}$.

Finally, for the third condition, note that each **repeat** iteration involves at most m loops and each loop introduces a single new oracle in the sequence. There are 4 of these loops for a single value of q and we iterate through $\lceil \frac{1}{\mu} \rceil$ values of q . Thus, we obtain that the total length of the sequence is bounded by $n < 24m/\mu$. $\square$

The rest of the proof follows in the same way as in the proof of Theorem 5.

Given that G queries the oracle at most T times, it cannot distinguish oracle access to $\mathcal{O}_i^j$ or $\mathcal{O}_{i+1}^j$ with better than $\frac{T}{m} < \frac{1}{\lambda_j}$ probability. Therefore, for any input $k \in \{0,1\}^{\lambda_j}$,

$$\text{Tr}(G^{\mathcal{O}_i^j}(k), G^{\mathcal{O}_{i+1}^j}(k)) \leq \frac{1}{\lambda_j}. \quad (28)$$

Now recall that there exists a set of inputs $\text{Disagr}_{\lambda_j}^{\mathcal{O}^j, \hat{\mathcal{O}}^j}$ where $G^{\mathcal{O}^j}$ and $G^{\hat{\mathcal{O}}^j}$ do not agree with at least $1/3$ probability.

For any input $k \in \text{Disagr}_{\lambda_j}^{\mathcal{O}^j, \hat{\mathcal{O}}^j}$ there are three cases that can occur:

1. There exists distinct values $y_k, \hat{y}_k$ such that

$$\Pr[G^{\mathcal{O}^j}(k) = y_k] \geq \frac{9}{10} \quad (29)$$

$$\Pr[G^{\hat{\mathcal{O}}^j}(k) = \hat{y}_k] \geq \frac{9}{10}. \quad (30)$$

Algorithm 1 Sequence of oracles $\mathcal{O}_1^j, \dots, \mathcal{O}_n^j$

- 1: Set $\mathcal{O}_1^j \leftarrow \mathcal{O}^j$, $P_1^j \leftarrow P^j$, and $Q_1^j \leftarrow Q^j$.
- 2: Set counter $i \leftarrow 2$. We inductively define the sequence of oracles as follows.
- 3: **for** $q \in \lceil [1/\mu] \rceil$ **do**
- 4: **repeat**
- 5: $(\mathcal{O}_i^j, P_i^j, Q_i^j)$ are defined the same as $(\mathcal{O}_{i-1}^j, P_{i-1}^j, Q_{i-1}^j)$, except for the following change: For every $x \in S_{T,q}$, set

$$Q_i^j(x) \leftarrow \max\left(Q_{i-1}^j(x) - \frac{1}{m}, 0\right).$$

- 6: $i \leftarrow i + 1$.
- 7: **until** we reach $Q_i^j(x) = 0$ for all $x \in S_{T,q}$.
- 8: Set $P_i^j(x)$ to 1 for all $x \in S_{T,q}$ and to $P_{i-1}^j(x)$ otherwise. Update $i \leftarrow i + 1$.
- 9: **repeat**
- 10: $(\mathcal{O}_i^j, P_i^j, Q_i^j)$ are defined the same as $(\mathcal{O}_{i-1}^j, P_{i-1}^j, Q_{i-1}^j)$, except for the following change: For every $x \in S_{T,q}$, set

$$Q_i^j(x) \leftarrow \min\left(Q_{i-1}^j(x) + \frac{1}{m}, 1\right).$$

- 11: $i \leftarrow i + 1$
- 12: **until** we reach $Q_i^j(x) = 1$ for all $x \in S_{T,q}$.
- 13: Set $\mathcal{O}_i^j(x)$ to $\hat{\mathcal{O}}^j(x)$ for all $x \in S_{T,q}$ and to $\mathcal{O}_{i-1}^j(x)$ otherwise. Update $i \leftarrow i + 1$.
- 14: **repeat**
- 15: $(\mathcal{O}_i^j, P_i^j, Q_i^j)$ are defined the same as $(\mathcal{O}_{i-1}^j, P_{i-1}^j, Q_{i-1}^j)$, except for the following change: For every $x \in S_{T,q}$, set

$$Q_i^j(x) \leftarrow \max\left(Q_{i-1}^j(x) - \frac{1}{m}, 0\right),$$

- 16: $i \leftarrow i + 1$
- 17: **until** we reach $Q_i^j(x) = 0$ for all $x \in S_{T,q}$.
- 18: Set $P_i^j(x)$ to $\hat{P}^j(x)$ for all $x \in S_{T,q}$ and to $P_{i-1}^j(x)$ otherwise. Update $i \leftarrow i + 1$.
- 19: **repeat**
- 20: $(\mathcal{O}_i^j, P_i^j, Q_i^j)$ are defined the same as $(\mathcal{O}_{i-1}^j, P_{i-1}^j, Q_{i-1}^j)$, except for the following change: For every $x \in S_{T,q}$, set

$$Q_i^j(x) \leftarrow \min\left(Q_{i-1}^j(x) + \frac{1}{m}, \hat{Q}^j(x)\right).$$

- 21: $i \leftarrow i + 1$
 - 22: **until** we reach $Q_i^j(x) = \hat{Q}^j(x)$ for all $x \in S_{T,q}$.
 - 23: **end for**
-

2. There does not exist a value y_k such that Eq. (29) holds. This implies that $G^{\mathcal{O}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic.
3. There does not exist a value $\hat{y}_k$ such that Eq. (30) holds. This implies that $G^{\hat{\mathcal{O}}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic.

We analyze each case separately.

- (Case 1): Consider the following probabilities:

$$p_{k,i} := \Pr \left[G^{\mathcal{O}_i^j}(k) = y_k \right] \quad (31)$$

Notice that, by Eqs. (29) and (30), we have $p_{k,1} \geq 9/10$ and $p_{k,n} \leq 1/10$. On the other hand, Eq. (28) states the output distribution under two consecutive oracles differs by at most $\frac{1}{\lambda_j}$. This means that there exists $i_k \in [m]$ such that

$$\frac{1}{4} \leq \frac{1}{2} - \frac{2}{\lambda_j} \leq \Pr \left[G^{\mathcal{O}_{i_k}^j}(k) = y_k \right] \leq \frac{1}{2} + \frac{2}{\lambda_j} \leq \frac{3}{4}. \quad (32)$$

In other words, $G^{\mathcal{O}_{i_k}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic.

- (Case 2): In this case, $G^{\mathcal{O}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic so set i_k to 1, noting that $\mathcal{O}^j = \mathcal{O}_1^j$.
- (Case 3): In this case, $G^{\hat{\mathcal{O}}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic so set i_k to n , noting that $\hat{\mathcal{O}}^j = \mathcal{O}_n^j$.

In other words, in all cases, there is an index i_k such that $G^{\mathcal{O}_{i_k}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic.

More generally, for every input in $\text{Disagr}_{\lambda_j}^{\mathcal{O}^j, \hat{\mathcal{O}}^j}$ there exists an oracle in the sequence under which the evaluation of G is $(\frac{1}{10})$ -non-deterministic.

The number of inputs in the set $\text{Disagr}_{\lambda_j}^{\mathcal{O}^j, \hat{\mathcal{O}}^j}$ is at least $\frac{2^{\lambda_j}}{\lambda_j}$. Therefore, there is an index, which we denote by i_j , such that there are at least $\frac{2^{\lambda_j}}{n\lambda_j}$ inputs $k \in \{0, 1\}^{\lambda_j}$, where $G^{\mathcal{O}_{i_j}^j}(k)$ is $(\frac{1}{10})$ -non-deterministic.

By performing this analysis for each $j \in \mathbb{N}$, we establish that there is some oracle $\bar{\mathcal{O}}^j$ such that $G^{\bar{\mathcal{O}}^j}$ is $(\frac{1}{10})$ -non-deterministic on at least $\frac{1}{n\lambda_j}$ fraction of inputs $k \in \{0, 1\}^{\lambda_j}$. Let $(\bar{\mathcal{O}}^j, \bar{P}^j, \bar{Q}^j)$ be the algorithms classifying the oracle $\bar{\mathcal{O}}^j$.

Assume without loss of generality, that for every $j \in \mathbb{N}$, $\lambda_{j+1} > 2^{2^{\lambda_j}}$. If the sequence does not satisfy this requirement, we can simply take a subsequence which does.

Notice that for any input $k_j \in \{0, 1\}^{\lambda_j}$, $G^{\mathcal{O}}(k_j)$ does not query the oracle on input of length less than $\log(\lambda_j)$ by definition, and since G is a QPT algorithm, it does not query on an input of length larger than 2^{λ_j} . Similarly, for any input $k_{j+1} \in \{0, 1\}^{\lambda_{j+1}}$, $G^{\mathcal{O}}(k_{j+1})$ does not query the oracle on input of length less than $\log(\lambda_{j+1}) > \log(2^{2^{\lambda_j}}) = 2^{\lambda_j}$, which is larger than the maximum query length when run on k_j . In other words, the queries of G do not intersect under different security parameters in the sequence $(\lambda_j)_{j \in \mathbb{N}}$.

This allows us to define the following oracle:

$$\bar{\mathcal{O}}(x) := \begin{cases} \bar{\mathcal{O}}^j(x) & x \in \{0, 1\}^d : \log(\lambda_j) \leq d \leq 2^{\lambda_j} \text{ for some } j \in \mathbb{N} \\ 0^{\ell(|x|)} & \text{otherwise} \end{cases} \quad (33)$$

Notice that for an input ρ of size $\log(\lambda_j) \leq |\rho| \leq 2^{\lambda_j}$, we have $\overline{\mathcal{O}}(\rho) = \overline{\mathcal{O}^j}(\rho)$. To summarize, for every $j \in \mathbb{N}$ we have

- $G^{\overline{\mathcal{O}^j}}$ is $(\frac{1}{10})$ -non-deterministic on $\frac{1}{n\lambda_j}$ fraction of inputs in $\{0, 1\}^{\lambda_j}$.
- For an input $k_j \in \{0, 1\}^{\lambda_j}$, $G^{\overline{\mathcal{O}^j}}(k_j)$ queries the oracle on inputs of length at most 2^{λ_j} and at least $\log(\lambda_j)$.
- For any state ρ of size $\log(\lambda_j) \leq |\rho| \leq 2^{\lambda_j}$, $\overline{\mathcal{O}}(\rho) = \overline{\mathcal{O}^j}(\rho)$.

Therefore, there is an infinite sequence $(\lambda_j)_{j \in \mathbb{N}}$ of security parameters such that, for each $j \in \mathbb{N}$, $G^{\overline{\mathcal{O}}}$ is $(\frac{1}{10})$ -non-deterministic on an inverse-polynomial fraction $(\frac{1}{\lambda_j^m})$ of inputs in $\{0, 1\}^{\lambda_j}$. This contradicts the correctness requirement for G since $\overline{\mathcal{O}} \in \mathcal{O}_{3\mu, \ell}$. $\square$

Define the oracle $\mathcal{Z}$ to be the element of $\mathcal{O}_{\mu, \ell}$ that maps any element $x \in \{0, 1\}^\lambda$ to $0^{\ell(\lambda)}$.

Consider the following hybrids of PRG security experiment.

- $\text{Exp}_1^A(\lambda)$:
 1. Sample oracle $\mathcal{O} \leftarrow \mathcal{O}_{\mu, \ell}$.
 2. Sample $k \leftarrow \{0, 1\}^\lambda$ and $b \leftarrow \{0, 1\}$.
 3. If $b = 0$, generate $y \leftarrow \overline{G}^{\mathcal{O}}(k)$. Else, sample $y \leftarrow \{0, 1\}^w$.
 4. $b' \leftarrow \mathcal{A}^T(y)$.
 5. If $b' = b$, output 1. Otherwise, output 0.
- $\text{Exp}_2^A(\lambda)$:
 1. Sample oracle $\mathcal{O} \leftarrow \mathcal{O}_{\mu, \ell}$.
 2. Sample $k \leftarrow \{0, 1\}^\lambda$ and $b \leftarrow \{0, 1\}$.
 3. If $b = 0$, generate $y \leftarrow \overline{G}^{\mathcal{O}}(k)$. Else, sample $y \leftarrow \{0, 1\}^w$.
 4. $b' \leftarrow \mathcal{A}^{\mathcal{T}_{\geq \log(\lambda)}}(y)$. Notice that $\mathcal{A}$ only has access to $\mathcal{T}_{\geq \log(\lambda)}$ in this variant.
 5. If $b' = b$, output 1. Otherwise, output 0.
- $\text{Exp}_3^A(\lambda)$:
 1. Sample oracle $\mathcal{O} \leftarrow \mathcal{O}_{\mu, \ell}$.
 2. Sample $k \leftarrow \{0, 1\}^{w-\lambda}$ and $b \leftarrow \{0, 1\}$.
 3. If $b = 0$, generate $y \leftarrow \overline{G}^{\mathcal{O}}(k)$. Else, sample $y \leftarrow \{0, 1\}^w$.
 4. $b' \leftarrow \mathcal{A}^{\mathcal{T}_{\geq \log(\lambda)}}(y)$.
 5. If $b' = b$, output 1. Otherwise, output 0.
- $\text{Exp}_4^A(\lambda)$:
 1. Sample oracle $\mathcal{O} \leftarrow \mathcal{O}_{\mu, \ell}$.
 2. Sample $k \leftarrow \{0, 1\}^{w-\lambda}$ and $b \leftarrow \{0, 1\}$.
 3. If $b = 0$, generate $y \leftarrow \overline{G}^{\mathcal{Z}}(k)$. Else, sample $y \leftarrow \{0, 1\}^w$.
 4. $b' \leftarrow \mathcal{A}^{\mathcal{T}_{\geq \log(\lambda)}}(y)$.
 5. If $b' = b$, output 1. Otherwise, output 0.
- $\text{Exp}_5^A(\lambda)$:
 1. Sample oracle $\mathcal{O} \leftarrow \mathcal{O}_{\mu, \ell}$.
 2. Sample $k \leftarrow \{0, 1\}^{w-\lambda}$ and $b \leftarrow \{0, 1\}$.

3. If $b = 0$, generate $y \leftarrow G^{\mathcal{Z}}(k)$. Else, sample $y \leftarrow \{0, 1\}^w$.
4. $b' \leftarrow \mathcal{A}^{\mathcal{C}}(y)$.
5. If $b' = b$, output 1. Otherwise, output 0.

By Claim 2, we have:

Claim 9. *For any QPT adversary $\mathcal{A}$ and large enough λ ,*

$$\Pr \left[\text{Exp}_1^{\mathcal{A}}(\lambda) = 1 \right] \leq \frac{1}{2} + \frac{1}{10}.$$

We now show that the experiment hybrids are indistinguishable except with small probability.

Claim 10. *For any QPT algorithm $\mathcal{A}$, there exists a QPT algorithm $\mathcal{B}$ such that*

$$\Pr \left[\text{Exp}_2^{\mathcal{A}}(\lambda) = 1 \right] \leq \Pr \left[\text{Exp}_1^{\mathcal{B}}(\lambda) = 1 \right]. \quad (34)$$

Proof. This is clear because the only difference between these experiments is that the adversary's oracle access is restricted. $\square$

Claim 11. *For any QPT algorithm $\mathcal{A}$, there exists a QPT algorithm $\mathcal{B}$ such that,*

$$\Pr \left[\text{Exp}_3^{\mathcal{B}}(\lambda) = 1 \right] \leq \Pr \left[\text{Exp}_2^{\mathcal{A}}(\lambda) = 1 \right] + \text{negl}(\lambda). \quad (35)$$

Proof. Notice that the only real difference between these hybrids is that in $\text{Exp}_2^{\mathcal{A}}(\lambda)$, the evaluation $\overline{G}^{\mathcal{O}}(k)$ uses $\mathcal{O}$ for all queries. Meanwhile, in $\text{Exp}_3^{\mathcal{B}}(\lambda)$, the evaluation of $G^{\mathcal{O}}(k)$ runs $\overline{G}$ but with oracle access to $\mathcal{O}$ restricted only on queries of length larger than $\log(\lambda)$, and for shorter queries, it uses an oracle simulated using the randomness of the input (see Fig. 2). This simulation is exact and so the output distributions of these two algorithms are indistinguishable. $\square$

Claim 12. *For any QPT algorithm $\mathcal{A}$ and large enough λ ,*

$$\Pr \left[\text{Exp}_4^{\mathcal{A}}(\lambda) = 1 \right] \leq \Pr \left[\text{Exp}_3^{\mathcal{A}}(\lambda) = 1 \right] + \frac{1}{3} + \frac{1}{\lambda}. \quad (36)$$

Proof. The only difference between these hybrids is G is given access to $\mathcal{Z}$ in $\text{Exp}_4^{\mathcal{A}}(\lambda)$ and is given access to $\mathcal{O}$ in $\text{Exp}_3^{\mathcal{A}}(\lambda)$.

By Lemma 4, there exists a constant λ^* such that for any $\lambda > \lambda^*$, we have

$$\Pr_{k \leftarrow \{0,1\}^\lambda} \left[k \in \text{Disagr}_{\lambda}^{\mathcal{Z}, \mathcal{O}} \right] < \frac{1}{\lambda}.$$

Therefore, there is at least $1 - \frac{1}{\lambda}$ probability that the key k sampled in the experiment does not belong to this set. In this case, we have

$$\Pr \left[G^{\mathcal{O}}(k) = G^{\mathcal{Z}}(k) \right] \geq 2/3. \quad (37)$$

Therefore, $\text{Exp}_3^A(\lambda)$ and $\text{Exp}_4^A(\lambda)$ can only be distinguished with at most

$$\left(1 - \frac{1}{\lambda}\right) \cdot \frac{1}{3} + \frac{1}{\lambda} \cdot 1 < \frac{1}{3} + \frac{1}{\lambda}$$

probability. $\square$

Claim 13. *For any QPT algorithm $\mathcal{A}$, there exists a QPT algorithm $\mathcal{B}$ such that*

$$\Pr \left[\text{Exp}_5^A(\lambda) = 1 \right] \leq \Pr \left[\text{Exp}_4^B(\lambda) = 1 \right]. \quad (38)$$

Proof. This is clear because the only difference between these experiments is that the adversary's oracle access is restricted. $\square$

As a result of Claims 9, 10, 11, 12, and 13, and the triangle inequality, we have for large enough λ ,

$$\Pr \left[\text{Exp}_5^A(\lambda) = 1 \right] \leq \frac{1}{2} + \frac{1}{10} + \frac{1}{3} + \frac{1}{\lambda} + \text{negl}(\lambda) = \frac{14}{15} + \frac{1}{\lambda} + \text{negl}(\lambda). \quad (39)$$

Notice that $\text{Exp}_5^A(\lambda)$ is just the PRG security experiment for G^Z against $\mathcal{A}^C$. However, G^Z can be easily simulated with a QPT algorithm without any oracle access since Z is just the zero function.

On the other hand, there exists a trivial search attack, using a PSPACE oracle, against PRG security, given that any polynomial-space quantum computations with classical inputs can be simulated using a PSPACE oracle. In particular, there exists an adversary $\bar{\mathcal{A}}$ such that

$$\Pr \left[\text{Exp}_5^{\bar{\mathcal{A}}}(\lambda) = 1 \right] \geq 1 - \text{negl}(\lambda).$$

contradicting Eq. (39) above.

Therefore, there does not exist a BB construction of a w -PRG from a $(3\mu, \ell)$ - $\perp$ -PRG. $\square$

References

1. Ananth, P., Lin, Y.T., Yuen, H.: Pseudorandom strings from pseudorandom quantum states. arXiv preprint arXiv:2306.05613 (2023)
2. Ananth, P., Qian, L., Yuen, H.: Cryptography from pseudorandom quantum states. In: Advances in Cryptology—CRYPTO 2022: 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15–18, 2022, Proceedings, Part I. pp. 208–236. Springer (2022)
3. Barhoush, M., Behera, A., Ozer, L., Salvail, L., Sattath, O.: Signatures from pseudorandom states via $\perp$ -prfs (2024), <https://arxiv.org/abs/2311.00847>
4. Barhoush, M., Nishimaki, R., Yamakawa, T.: Microcrypt assumptions with quantum input sampling and pseudodeterminism: Constructions and separations. arXiv preprint arXiv:2505.14461 (2025)

5. Bartusek, J.: Secure quantum computation with classical communication. In: Theory of Cryptography Conference. pp. 1–30. Springer (2021)
6. Blum, M., Micali, S.: How to generate cryptographically strong sequences of pseudorandom bits. *SIAM Journal on Computing* **13**(4), 850–864 (1984). <https://doi.org/10.1137/0213053>, <https://doi.org/10.1137/0213053>
7. Bouaziz-Ermann, S., Hhan, M., Muguruza, G., Vu, Q.H.: On limits on the provable consequences of quantum pseudorandomness. *Cryptology ePrint Archive*, Paper 2025/1863 (2025), <https://eprint.iacr.org/2025/1863>
8. Bouaziz-Ermann, S., Muguruza, G.: Quantum pseudorandomness cannot be shrunk in a black-box way (2024), <https://doi.org/10.48550/arXiv.2402.13324>
9. Brakerski, Z., Shmueli, O.: Scalable pseudorandom quantum states. In: Annual International Cryptology Conference. pp. 417–440. Springer (2020)
10. Brakerski, Z., Shmueli, O.: Scalable pseudorandom quantum states. In: Annual International Cryptology Conference. pp. 417–440. Springer (2020)
11. Chen, B., Coladangelo, A., Sattath, O.: The power of a single haar random state: constructing and separating quantum pseudorandomness. *arXiv preprint arXiv:2404.03295* (2024)
12. Coladangelo, A., Mutreja, S.: On black-box separations of quantum digital signatures from pseudorandom states. In: Theory of Cryptography Conference. pp. 289–317. Springer (2024)
13. Impagliazzo, R., Rudich, S.: Limits on the provable consequences of one-way permutations. In: Proceedings of the twenty-first annual ACM symposium on Theory of computing. pp. 44–61 (1989)
14. Ji, Z., Liu, Y.K., Song, F.: Pseudorandom quantum states. In: Advances in Cryptology–CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part III 38. pp. 126–152. Springer (2018)
15. Kretschmer, W.: Quantum pseudorandomness and classical complexity. In: 16th Conference on the Theory of Quantum Computation, Communication and Cryptography (2021)
16. Kretschmer, W., Qian, L., Tal, A.: Quantum-computable one-way functions without one-way functions. *arXiv preprint arXiv:2411.02554* (2024)
17. Mahmoody, M., Mohammed, A., Nematihaji, S., Pass, R., et al.: A note on black-box separations for indistinguishability obfuscation. *Cryptology ePrint Archive* (2016)
18. Meckes, E.S.: The random matrix theory of the classical compact groups, vol. 218. Cambridge University Press (2019)
19. Morimae, T., Yamakawa, T.: Quantum commitments and signatures without one-way functions. In: Advances in Cryptology–CRYPTO 2022: 42nd Annual International Cryptology Conference, CRYPTO 2022, Santa Barbara, CA, USA, August 15–18, 2022, Proceedings, Part I. pp. 269–295. Springer (2022)
20. Nielsen, M.A., Chuang, I.L.: Quantum computation and quantum information. Cambridge University Press, New York, NY, USA (2000). <https://doi.org/10.1017/CB09780511976667>
21. Saito, T., Xagawa, K., Yamakawa, T.: Tightly-secure key-encapsulation mechanism in the quantum random oracle model. In: Advances in Cryptology–EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29–May 3, 2018 Proceedings, Part III 37. pp. 520–551. Springer (2018)